

LAPORAN PRAKTIKUM PEKAN 7



MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU :

Dr. WAHYUDI, S.T., M.T.

OLEH :

MUHAMMAD HANS NAFIS

NIM : 2511532027

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kepada Allah SWT atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan Laporan Praktikum Pekan 7 ini tepat pada waktunya. Laporan ini disusun untuk memenuhi salah satu tugas praktikum mata kuliah Basis Data, dengan topik pembahasan mengenai konsep dasar proses *sorting* (*Insertion Sort*, *Selection Sort*, *Bubble Sort*).

Penyusunan laporan ini bertujuan untuk mendokumentasikan hasil praktikum yang telah dilakukan, sekaligus sebagai saran untuk memperdalam pemahaman penulis terhadap materi yang telah dipelajari selama kegiatan praktikum berlangsung. Dalam proses penyusunan laporan ini, penulis memperoleh banyak pengalaman dan pengetahuan baru, khususnya dalam memahami konsep dasar proses *sorting* serta penerapannya dalam pemograman menggunakan bahasa Java.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Akhir kata, penulis mengucapkan terima kasih kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Struktur Data, asisten dosen, teman-teman praktikum dan pihak lain yang turut mendukung penulisan laporan ini.

Selasa, 19 Mei 2026

Muhammad Hans Nafis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB 1	1
PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB 2	2
PEMBAHASAN	2
2.1 <i>Sorting</i>	2
2.2 Class InsertionSort.....	2
2.3 Class SelectionSort.....	4
2.4 Class BubbleSort.....	6
2.5 Class InsertionSortGUI	8
2.6 Tugas Praktikum Pekan 7	15
BAB 3	29
KESIMPULAN.....	29
DAFTAR PUSTAKA	30
LAMPIRAN.....	31

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman dan pengolahan data, proses pengurutan data merupakan salah satu hal yang sangat penting. Data yang telah terurut akan mempermudah proses pencarian, pengelompokan, serta pengolahan informasi lainnya. Oleh karena itu, diperlukan pemahaman mengenai berbagai metode *sorting* agar programmer dapat memilih logaritma yang sesuai dengan kebutuhan.

Terdapat berbagai macam algoritma pengurutan data, diantaranya adalah *Insertion Sort*, *Selection Sort*, dan *Bubble Sort*. Ketiga metode tersebut memiliki cara kerja yang berbeda dalam mengurutkan data, namun sama-sama bertujuan menghasilkan data yang tersusun secara teratur, baik secara *ascending* maupun *descending*.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut :

1. Memahami konsep dasar algoritma pengurutan data (*sorting*).
2. Mengetahui cara kerja algoritma *Insertion Sort*.
3. Mengetahui cara kerja algoritma *Selection Sort*.
4. Mengetahui cara kerja algoritma *Bubble Sort*.

1.3 Manfaat Praktikum

Manfaat dari pelaksanaan praktikum antara lain sebagai berikut :

1. Menambah pemahaman mengenai teknik pengurutan data.
2. Membantu memahami perbedaan proses kerja setiap algoritma *sorting*.
3. Meningkatkan kemampuan pemrograman dalam bahasa Java.
4. Melatih keterampilan dalam mengelola kode program secara terstruktur.

BAB 2

PEMBAHASAN

2.1 *Sorting*

Sorting merupakan proses mengurutkan data ke dalam urutan tertentu, seperti *ascending* atau *descending*. Pengurutan data sangat penting dalam pemrograman karena dapat mempermudah proses pencarian data, pengolahan data, serta meningkatkan efisiensi program. Terdapat berbagai macam algoritma pengurutan data, diantaranya adalah *Insertion Sort*, *Selection Sort*, dan *Bubble Sort*.

Insertion Sort melakukan pengurutan dengan cara menyisipkan elemen ke posisi yang tepat pada bagian data yang sudah terurut. Algoritma ini bekerja mirip seperti menyusun kartu secara manual, yaitu mengambil satu data kemudian menempatkannya pada posisi yang sesuai. *Insertion Sort* umumnya lebih efisien ketika data yang diurutkan berjumlah kecil atau hampir terurut. Hal ini karena jumlah perpindahan dan perbandingan data yang dilakukan relatif lebih sedikit.

Selection Sort bekerja dengan cara mencari nilai terkecil dari bagian *array* yang belum terurut, kemudian menukarnya dengan elemen pada posisi awal bagian tersebut. Setelah satu data ditempatkan pada posisi yang benar, proses dilanjutkan untuk bagian data berikutnya hingga seluruh data terurut. Metode ini memiliki jumlah pertukaran data yang lebih sedikit dibanding *Bubble Sort*, sehingga dalam beberapa kondisi dapat bekerja dengan cepat. Akan tetapi, proses pencarian nilai minimum pada setiap iterasi tetap membuat kompleksitas waktunya cukup besar.

Bubble Sort bekerja dengan cara membandingkan dua elemen yang berdekatan, kemudian menukarnya apabila urutan salah. Proses ini dilakukan secara berulang hingga seluruh data tersusun dengan benar. Pada setiap iterasi, nilai terbesar atau terkecil akan berpindah ke posisi akhirnya, sehingga prosesnya menyerupai gelembung yang naik ke permukaan. *Bubble Sort* memiliki konsep yang dilakukan secara terus-menerus menyebabkan algoritma ini kurang efisien untuk data dalam jumlah besar.

2.2 Class InsertionSort

Berikut kode program Java mengenai class InsertionSort :

```
public class InsertionSort_2511532027 {  
    public static void insertionSort_2027(int[] arr_2027) {  
        int n_2027 = arr_2027.length;
```

```

        for (int i_2027 = 1; i_2027 < n_2027; i_2027++) {
            int key_2027 = arr_2027[i_2027];
            int j_2027 = i_2027 - 1;
            while (j_2027 >= 0 && arr_2027[j_2027] > key_2027) {
                arr_2027[j_2027 + 1] = arr_2027[j_2027];
                j_2027--;
            }
            arr_2027[j_2027 + 1] = key_2027;
        }
    }

    public static void main (String[] args) {
        int arr_2027[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_2027 = arr_2027.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i_2027 = 0; i_2027 < n_2027; i_2027++)
            System.out.print(arr_2027[i_2027] + " ");
        System.out.println("");
        insertionSort_2027(arr_2027);
        System.out.printf("array yang terurut:\n");
        for (int i_2027 = 0; i_2027 < n_2027; i_2027++)
            System.out.print(arr_2027[i_2027] + " ");
        System.out.println("");
    }
}

```

Kode program tersebut digunakan untuk mengimplementasikan algoritma *Insertion Sort* dalam proses pengurutan data *array* secara *ascending*. Program terdiri dari kelas `InsertionSort_2511532027` yang memiliki method `insertionSort_2027()` dan method utama `main()`. Method `insertionSort_2027()` berfungsi untuk melakukan proses pengurutan data, sedangkan method `main()` digunakan untuk menjalankan program dan menampilkan hasil sebelum serta sesudah data diurutkan.

Pada method `insertionSort_2027()`, program menentukan jumlah elemen *array* menggunakan `arr_2027.length` dan menyimpannya ke dalam variabel `n_2027`. Selanjutnya digunakan perulangan `for` yang dimulai dari indeks ke-1 karena elemen pertama dianggap sudah terurut. Variabel `key_2027` digunakan

untuk menyimpan nilai yang akan dibandingkan dan disisipkan ke posisi yang tepat. Variabel `j_2027` digunakan untuk menelusuri elemen sebelumnya pada *array*.

Proses utama algoritma terjadi pada perulangan `while`, yaitu ketika elemen sebelumnya lebih besar daripada nilai `key_2027`. Jika kondisi tersebut terpenuhi, maka elemen sebelumnya akan digeser satu posisi ke kanan. Proses ini terus dilakukan hingga ditemukan posisi yang sesuai untuk menyisipkan nilai `key_2027`. Setelah posisi yang tepat ditemukan, nilai `key_2027` dimasukkan ke dalam *array* menggunakan perintah `arr_2027[j_2027 + 1] = key_2027`. Dengan cara ini, data akan tersusun secara berurutan dari nilai terkecil ke terbesar.

Pada method `main()`, program mendeklarasikan *array* berisi beberapa angka, yaitu 23, 78, 45, 8, 32, 56, dan 1. Program kemudian menampilkan isi *array* sebelum diurutkan menggunakan perulangan `for`. Setelah itu, method `insertionSort_2027()` dipanggil untuk mengurutkan data. Hasil *array* yang telah terurut kemudian ditampilkan kembali ke layar.

2.3 Class SelectionSort

Berikut kode program Java mengenai class `SelectionSort` :

```
public class SelectionSort_2511532027 {
    public static void selectionSort_2027(int[] arr_2027) {
        int n_2027 = arr_2027.length;
        for (int i_2027 = 0; i_2027 < n_2027; i_2027++) {
            int minIndex_2027 = i_2027;
            for (int j_2027 = i_2027 + 1; j_2027 < n_2027;
j_2027++) {
                if (arr_2027[j_2027] <
arr_2027[minIndex_2027]) {
                    minIndex_2027 = j_2027;
                }
            }
            int temp_2027 = arr_2027[i_2027];
            arr_2027[i_2027] = arr_2027[minIndex_2027];
            arr_2027[minIndex_2027] = temp_2027;
        }
    }
    public static void main (String[] args) {
        int arr_2027[] = { 23, 78, 45, 8, 32, 56, 1 };
    }
}
```

```

        int n_2027 = arr_2027.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i_2027 = 0; i_2027 < n_2027; i_2027++)
            System.out.print(arr_2027[i_2027] + " ");
        System.out.println("");
        selectionSort_2027(arr_2027);
        System.out.printf("array yang terurut:\n");
        for (int i_2027 = 0; i_2027 < n_2027; i_2027++)
            System.out.print(arr_2027[i_2027] + " ");
        System.out.println("");
    }
}

```

Kode program tersebut digunakan untuk mengimplementasikan algoritma *Selection Sort* dalam proses pengurutan data *array* secara *ascending*. Program terdiri dari kelas `SelectionSort_2511532027` yang memiliki method `selectionSort_2027()` untuk melakukan proses *sorting* dan method `main()` untuk menjalankan program. Algoritma *Selection Sort* bekerja dengan cara mencari nilai terkecil dari bagian *array* yang belum terurut, kemudian menukarkannya dengan elemen pada posisi awal sehingga data secara bertahap menjadi terurut.

Pada method `selectionSort_2027()`, program menentukan jumlah elemen *array* menggunakan `arr_2027.length` dan menyimpannya ke dalam variabel `n_2027`. Selanjutnya digunakan perulangan `for` pertama dengan variabel `i_2027` untuk menentukan posisi data yang akan diisi dengan nilai terkecil. Variabel `minIndex_2027` digunakan untuk menyimpan indek dari nilai terkecil yang ditemukan selama proses pencarian. Awalnya, indeks minimum diisi dengan nilai indeks saat ini (`i_2027`).

Kemudian program menjalankan perulangan `for` kedua menggunakan variabel `j_2027` untuk mencari nilai terkecil pada bagian *array* yang belum terurut. Pada proses ini dilakukan pengecekan menggunakan kondisi `if`, yaitu apabila elemen `arr_2027[j_2027]` lebih kecil daripada elemen pada indeks minimum (`arr_2027[minIndex_2027]`), maka indeks minimum akan diperbarui sesuai posisi elemen terkecil tersebut. Setelah seluruh proses pencarian selesai, program melakukan pertukaran data menggunakan variabel sementara `temp_2027`. Nilai

pada posisi awal ditukar dengan nilai terkecil yang telah ditemukan sehingga elemen terkecil berpindah ke posisi yang benar.

Pada method `main()`, program mendeklarasikan *array* yang berisikan beberapa angka, yaitu 23, 78, 45, 8, 32, 56, dan 1. Program kemudian menampilkan isi *array* sebelum proses *sorting* dilakukan menggunakan perulangan `for`. Setelah itu method `selectionSort_2027()` dipanggil untuk mengurutkan data. Setelah proses pengurutan selesai, program kembali menampilkan isi *array* yang sudah terurut secara *ascending*.

2.4 Class BubbleSort

Berikut kode program Java mengenai class `BubbleSort` :

```
public class BubbleSort_2511532027 {
    public static void bubbleSort_2027 (int[] arr_2027) {
        int n_2027 = arr_2027.length;
        for (int i_2027 = 0; i_2027 < n_2027; i_2027++) {
            for (int j_2027 = 0; j_2027 < n_2027 - i_2027 - 1;
j_2027++) {
                if (arr_2027[j_2027] > arr_2027[j_2027 + 1])
{
                    int temp_2027 = arr_2027[j_2027];
                    arr_2027[j_2027] = arr_2027[j_2027 +
1];
                    arr_2027 [j_2027 + 1] = temp_2027;
                    //
System.out.println("data:"+arr_2027[j_2027]+" "+arr_2027[j_2027+1]);
                }
            }
        }
    }
    public static void main (String[] args) {
        int arr_2027[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_2027 = arr_2027.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i_2027 = 0; i_2027 < n_2027; i_2027++)
            System.out.print(arr_2027[i_2027] + " ");
        System.out.println("");
        bubbleSort_2027(arr_2027);
    }
}
```

```

        System.out.printf("array yang terurut:\n");
        for (int i_2027 = 0; i_2027 < n_2027; i_2027++)
            System.out.print(arr_2027[i_2027] + " ");
        System.out.println("");
    }
}

```

Kode program tersebut digunakan untuk mengimplementasikan algoritma *Bubble Sort* dalam proses pengurutan data *array* secara *ascending*. Program terdiri dari kelas `BubbleSort_2511532027` yang memiliki method `bubbleSort_2027()` untuk melakukan proses pengurutan data dan method `main()` untuk menjalankan program. Algoritma *Bubble Sort* bekerja dengan cara membandingkan dua elemen yang berdekatan, kemudian menukarnya apabila urutannya salah. Proses ini dilakukan secara berulang hingga seluruh data tersusun secara teratur.

Pada method `bubbleSort_2027()`, program menentukan jumlah elemen *array* menggunakan `arr_2027.length` dan menyimpannya ke dalam variabel `n_2027`. Selanjutnya digunakan dua buah perulangan `for`. Perulangan pertama dengan variabel `i_2027` berfungsi untuk menentukan jumlah tahapan pengurutan yang dilakukan. Sedangkan perulangan kedua dengan variabel `j_2027` digunakan untuk membandingkan elemen-elemen *array* yang berdekatan. Batas perulangan kedua menggunakan `n_2027 - i_2027 - 1` karena pada setiap iterasi, elemen terbesar sudah berada pada posisi akhir sehingga tidak perlu dibandingkan kembali.

Di dalam perulangan kedua terdapat kondisi `if` yang digunakan untuk mengecek apakah elemen pada indeks saat ini lebih besar daripada elemen berikutnya. Jika kondisi tersebut terpenuhi, maka program akan melakukan pertukaran data menggunakan variabel sementara `temp_2027`. Nilai elemen pertama dipindahkan ke variabel sementara, kemudian elemen kedua dipindahkan ke posisi pertama, dan nilai sementara ditempatkan pada posisi kedua. Dengan proses penukaran ini, nilai yang lebih besar secara bertahap akan berpindah ke bagian akhir *array* seperti gelembung yang naik ke permukaan, sehingga metode ini disebut *Bubble Sort*.

Pada method `main()`, program mendeklarasikan *array* berisi beberapa angka, yaitu 23, 78, 45, 8, 32, 56, dan 1. Program kemudian menampilkan isi *array* sebelum diurutkan menggunakan perulangan `for`. Setelah itu method

bubbleSort_2027() dipanggil untuk melakukan proses pengurutan data. Setelah proses *sorting* selesai, program kembali menampilkan isi *array* yang telah terurut secara *ascending*.

2.5 Class InsertionSortGUI

Berikut kode program Java mengenai class InsertionSortGUI :

```
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;

public class InsertionSortGUI_2511532027 extends JFrame {
    private static final long serialVersionUID = 1L;
    private int[] array_2027;
    private JLabel[] labelArray_2027;
    private JButton stepButton_2027, resetButton_2027,
setButton_2027;
    private JTextField inputField_2027;
    private JPanel panelArray_2027;
    private JTextArea stepArea_2027;
    private int i_2027 = 1, j_2027;
    private boolean sorting_2027 = false;
    private int stepCount_2027 = 1;

    /**
     * Create the frame.

```

```

*/
public InsertionSortGUI_2511532027() {
    setTitle("Insertion Sort Langkah per Langkah");
    setSize(750, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());

    //Panel input
    JPanel inputPanel_2027 = new JPanel (new FlowLayout());
    inputField_2027 = new JTextField(30);
    setButton_2027 = new JButton("Set Array");
    inputPanel_2027.add(new JLabel("Masukkan angka (pisahkan
dengan koma:")));
    inputPanel_2027.add(inputField_2027);
    inputPanel_2027.add(setButton_2027);

    // Panel array visual
    panelArray_2027 = new JPanel();
    panelArray_2027.setLayout(new FlowLayout());

    // Panel kontrol
    JPanel controlPanel_2027 = new JPanel();
    stepButton_2027 = new JButton("Langkah Selanjutnya");
    resetButton_2027 = new JButton("Reset");
    stepButton_2027.setEnabled(false);
    controlPanel_2027.add(stepButton_2027);
    controlPanel_2027.add(resetButton_2027);

    // Area teks untuk log langkah-langkah
    stepArea_2027 = new JTextArea(8, 60);
    stepArea_2027.setEditable(false);
    stepArea_2027.setFont(new Font("Monospaced", Font.PLAIN,
14));

    JScrollPane scrollPane_2027 = new
JScrollPane(stepArea_2027);

    // Tambahkan panel ke frame

```

```

        add(inputPanel_2027, BorderLayout.NORTH);
        add(panelArray_2027, BorderLayout.CENTER);
        add(controlPanel_2027, BorderLayout.SOUTH);
        add(scrollPane_2027, BorderLayout.EAST);

        // Event Set Array
        setButton_2027.addActionListener(e_2027 ->
setArrayFromInput_2027());

        // Event Langkah Selanjutnya
        stepButton_2027.addActionListener(e_2027 ->
performStep_2027());

        // Event Reset
        resetButton_2027.addActionListener(e_2027 ->
reset_2027());}

        private void setArrayFromInput_2027() {
            String text_2027 = inputField_2027.getText().trim();
            if (text_2027.isEmpty()) return;
            String[] parts_2027 = text_2027.split(",");
            array_2027 = new int[parts_2027.length];
            try {
                for (int k_2027 = 0; k_2027 <
parts_2027.length; k_2027++) {
                    array_2027[k_2027] =
Integer.parseInt(parts_2027[k_2027].trim());
                }} catch (NumberFormatException e_2027) {
                    JOptionPane.showMessageDialog(this,
"Masukkan hanya angka yang dipisahkan "
+ "dengan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
                return; }
            i_2027 = 1;
            stepCount_2027 = 1;
            sorting_2027 = true;
            stepButton_2027.setEnabled(true);
            stepArea_2027.setText("");

```

```

        panelArray_2027.removeAll();
        labelArray_2027 = new
JLabel[array_2027.length];
        for (int k_2027 = 0; k_2027 <
array_2027.length; k_2027++) {
            labelArray_2027[k_2027] = new
JLabel(String.valueOf(array_2027[k_2027]));
            labelArray_2027[k_2027].setFont(new
Font("Arial", Font.BOLD, 24));

            labelArray_2027[k_2027].setBorder(BorderFactory.createLineBorder(
Color.BLACK));

            labelArray_2027[k_2027].setPreferredSize(new Dimension(50, 50));

            labelArray_2027[k_2027].setHorizontalAlignment(SwingConstants.CEN
TER);

            panelArray_2027.add(labelArray_2027[k_2027]);
        }
        panelArray_2027.revalidate();
        panelArray_2027.repaint();
    }
    private void performStep_2027() {
        if (i_2027 < array_2027.length &&
sorting_2027) {

            int key_2027 = array_2027[i_2027];
            j_2027 = i_2027 - 1;

            StringBuilder stepLog_2027 = new
StringBuilder();

            stepLog_2027.append("Langkah
").append(stepCount_2027).
            append(": Memasukkan
").append(key_2027).append("\n");

            while (j_2027 >= 0 &&
array_2027[j_2027] > key_2027) {

```

```

        array_2027[j_2027 + 1] =
array_2027[j_2027];

        j_2027--;
    }
    array_2027[j_2027 + 1] = key_2027;

    updateLabels_2027();
    stepLog_2027.append("Hasil:
").append(arrayToString_2027(array_2027)).append("\n\n");

    stepArea_2027.append(stepLog_2027.toString());

    i_2027++;
    stepCount_2027++;

    if (i_2027 == array_2027.length) {
        sorting_2027 = false;

    stepButton_2027.setEnabled(false);

    JOptionPane.showMessageDialog(this, "Sorting selesai!");
    }
}

private void updateLabels_2027() {
    for (int k_2027 = 0; k_2027 <
array_2027.length; k_2027++) {

        labelArray_2027[k_2027].setText(String.valueOf(array_2027[k_2027]
));

    }
}

private void reset_2027() {
    inputField_2027.setText("");
    panelArray_2027.removeAll();
    panelArray_2027.revalidate();
    panelArray_2027.repaint();
    stepArea_2027.setText("");
}

```

```

        stepButton_2027.setEnabled(false);
        sorting_2027 = false;
        i_2027 = 1;
        stepCount_2027 = 1;
    }
    private String arrayToString_2027(int[] arr_2027) {
        StringBuilder sb_2027 = new StringBuilder();
        for (int k_2027 = 0; k_2027 <
arr_2027.length; k_2027++) {
            sb_2027.append(arr_2027[k_2027]);
            if (k_2027 < arr_2027.length - 1)
sb_2027.append(", ");
        }
        return sb_2027.toString();
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            InsertionSortGUI_2511532027 gui_2027 =
new InsertionSortGUI_2511532027();
            gui_2027.setVisible(true);
        });
    }
}

```

Kode program tersebut merupakan implementasi algoritma *Insertion Sort* berbasis GUI menggunakan library Swing. Program ini dibuat untuk menampilkan proses pengurutan data secara langkah demi langkah sehingga pengguna dapat melihat bagaimana algoritma *Insertion Sort* bekerja secara visual. Kelas `InsertionSortGUI_2511532027` merupakan turunan dari `JFrame`, sehingga program memiliki tampilan jendela aplikasi lengkap dengan tombol, area input, dan tampilan *array*. Program ini juga menggunakan beberapa komponen GUI seperti `JButton`, `JTextField`, `JLabel`, `JPanel`, dan `JTextArea` untuk mendukung interaksi pengguna.

Pada konstruktor, program mengatur tampilan utama aplikasi seperti judul *frame*, ukuran jendela, posisi tampilan, dan *layout* menggunakan `BorderLayout`. Selanjutnya dibuat panel input yang berisi `JTextField` untuk memasukkan data *array* dan tombol `Set Array` untuk memproses input pengguna. Data angka dimasukkan dengan format dipisahkan menggunakan tanda koma. Program juga

membuat panel visual *array* yang digunakan untuk menampilkan elemen-elemen *array* dalam bentuk label sehingga pengguna dapat melihat perubahan data selama proses *sorting* berlangsung.

Program juga memiliki panel kontrol yang terdiri dari tombol Langkah Selanjutnya dan Reset. Tombol Langkah Selanjutnya digunakan untuk menjalankan proses *Insertion Sort* satu langkah setiap kali tombol ditekan, sedangkan tombol Reset digunakan untuk mengembalikan program ke kondisi awal. Selain itu terdapat JTextArea yang digunakan sebagai tempat menampilkan *log* atau catatan proses *sorting* sehingga pengguna dapat melihat hasil setiap langkah pengurutan secara detail. Area teks tersebut dibungkus menggunakan JScrollPane agar dapat digulir ketika isi teks terlalu panjang.

Method `setArrayFromInput_2027()` berfungsi untuk mengambil input angka dari pengguna, kemudian memisahkan data menggunakan fungsi `split(",")`. Setiap data kemudian dikonversi menjadi tipe integer menggunakan `Integer.parseInt()`. Jika pengguna memasukkan data selain angka, program akan menampilkan pesan error menggunakan JOptionPane. Setelah data berhasil diproses, program membuat label-label *array* menggunakan JLabel untuk menampilkan setiap elemen *array* secara visual pada panel utama. Label tersebut diberi border, ukuran tertentu, dan posisi teks di tengah agar tampil lebih rapi.

Proses utama algoritma *Insertion Sort* dilakukan pada method `performStep_2027()`. Method ini bekerja ketika tombol Langkah Selanjutnya ditekan. Program mengambil nilai pada indeks tertentu sebagai `key_2027`, kemudian membandingkannya dengan elemen sebelumnya. Jika elemen sebelumnya lebih besar, maka data akan digeser satu posisi ke kanan hingga ditemukan posisi yang sesuai untuk menyisipkan nilai `key_2027`. Setelah proses selesai, method `updateLabels_2027()` dipanggil untuk memperbarui tampilan *array* pada GUI sehingga pengguna dapat melihat hasil pengurutan secara langsung. Program juga mencatat setiap langkah *sorting* ke dalam `stepArea_2027` agar proses pengurutan dapat dipahami dengan lebih mudah.

Method `reset_2027()` digunakan untuk menghapus seluruh data input, membersihkan panel *array*, menghapus *log* langkah *sorting*, dan mengembalikan nilai variabel ke kondisi awal. Sedangkan method `arrayToString_2027()` digunakan

untuk mengubah isi *array* menjadi bentuk string agar dapat ditampilkan pada area *log*. Pada method *main()*, program dijalankan menggunakan *SwingUtilities.invokeLater()* untuk memastikan seluruh komponen GUI dibuat dan dijalankan pada *thread* Swing yang benar.

2.6 Tugas Praktikum Pekan 7

Berikut tugas praktikum yang diberikan :

Pengembangan Tugas Praktikum Pekan 7: Struktur Data

Topik : Algoritma Sorting (Insertion Sort, Selection Sort, dan Bubble Sort)

Tema : Pengurutan Nama Mahasiswa Secara Alfabetis Menggunakan GUI

1. Deskripsi Program

Program dikembangkan menggunakan Java GUI (Swing) untuk menampilkan proses pengurutan data mahasiswa secara interaktif.

Program menyediakan beberapa fitur utama, yaitu:

- Input data mahasiswa secara manual melalui form GUI.
- Menyimpan data mahasiswa ke dalam array atau ArrayList object.
- Pemilihan algoritma sorting menggunakan ComboBox.
- Menampilkan visualisasi proses sorting langkah demi langkah.
- **Menampilkan hasil akhir pengurutan nama mahasiswa secara ascending (A-Z).**

Perbandingan string dilakukan menggunakan method *compareToIgnoreCase()* agar proses sorting tidak membedakan huruf kapital dan huruf kecil.

2. Struktur Program

Program terdiri dari beberapa bagian utama:

A. Kelas ADT Mahasiswa

Class digunakan untuk menyimpan data mahasiswa.

Atribut:

- Nama Mahasiswa (String)
- NIM Mahasiswa (String)
- Program Studi (String)

Method:

- Constructor
- Getter dan Setter

Gambar 2.1

- toString()

B. Kelas GUI Utama

Class GUI digunakan untuk:

- Menampilkan form input data.
- Menampilkan tabel data mahasiswa.
- Menampilkan ComboBox pilihan algoritma sorting.
- Menampilkan proses visualisasi sorting.
- Menjalankan algoritma sorting sesuai pilihan pengguna.

C. Method Sorting

Program menyediakan tiga method sorting terpisah:

- insertionSort()
- selectionSort()
- bubbleSort()

3. Komponen GUI

GUI program terdiri dari beberapa komponen berikut:

- JTextField

Digunakan untuk input nama mahasiswa, NIM, dan program studi.

- JButton

Digunakan untuk:

- Menambahkan data
- Menghapus data
- Memulai sorting

- JComboBox

Digunakan untuk memilih algoritma sorting:



Gambar 2.2

- Insertion Sort
- Selection Sort
- Bubble Sort

- JTable / JTextArea

Digunakan untuk menampilkan:

- Data mahasiswa
- Langkah-langkah proses sorting
- Hasil akhir sorting

4. Alur Program

1. Pengguna menginput data mahasiswa melalui GUI.
2. Data disimpan ke dalam array atau ArrayList.
3. Pengguna memilih algoritma sorting pada ComboBox.
4. Pengguna menekan tombol "Mulai Sorting".
5. Program menjalankan algoritma sorting yang dipilih.
6. Proses sorting ditampilkan visualisasinya langkah demi langkah.
7. Hasil akhir pengurutan ditampilkan pada GUI.

5. Visualisasi Sorting (menggunakan Java GUI (Swing))

Setiap algoritma sorting wajib menampilkan proses pengurutan secara bertahap agar alur algoritma dapat dipahami dengan jelas.

Contoh visualisasi (Java GUI (Swing))

=== INSERTION SORT ===

Langkah 1 : [Budi, Rizky, Maya]

Langkah 2 : [Budi, Maya, Rizky]



=== SELECTION SORT ===

Pass 1 : [Andi, Budi, Maya, Rizky]

Gambar 2.3

=== BUBBLE SORT ===

Pass 1 : [Budi, Maya, Andi, Rizky]

6. Kriteria Penilaian

- Ketepatan implementasi dan pemahaman logika ketiga algoritma sorting untuk data bertipe String.
- Penggunaan `compareToIgnoreCase()` untuk perbandingan string.
- Kesesuaian aturan penamaan kelas dan variabel.
- Visualisasi proses tiap langkah sorting ditampilkan dengan jelas di console.
- Data awal yang digunakan identik untuk ketiga algoritma.
- Kerapihan kode dan pemberian komentar pada bagian logika sorting.
- Project berhasil di-push ke GitHub melalui Eclipse (EGit).

Gambar 2.4

Berikut kode program Java dari tugas praktikum :

```
public class Mahasiswa_2511532027 {

    private String nama_2027;
    private String nim_2027;
    private String prodi_2027;

    // Constructor
    public Mahasiswa_2511532027(
        String nama_2027,
        String nim_2027,
        String prodi_2027) {

        this.nama_2027 = nama_2027;
        this.nim_2027 = nim_2027;
        this.prodi_2027 = prodi_2027;
    }

    // Getter dan Setter
    public String getNama_2027() {
        return nama_2027;
    }

    public void setNama_2027(String nama_2027) {
        this.nama_2027 = nama_2027;
    }
}
```

```

    }

    public String getNim_2027() {
        return nim_2027;
    }

    public void setNim_2027(String nim_2027) {
        this.nim_2027 = nim_2027;
    }

    public String getProdi_2027() {
        return prodi_2027;
    }

    public void setProdi_2027(String prodi_2027) {
        this.prodi_2027 = prodi_2027;
    }

    @Override
    public String toString() {
        return nama_2027;
    }
}

```

Kode program tersebut merupakan kelas ADT bernama Mahasiswa_2511532027 yang digunakan untuk menyimpan dan mengelola data mahasiswa. Kelas ini berfungsi sebagai representasi objek mahasiswa yang memiliki beberapa atribut penting, yaitu nama mahasiswa, NIM, dan program studi. Ketiga atribut tersebut bertipe data String dan bersifat private, sehingga data hanya dapat diakses melalui method tertentu. Penggunaan *access modifier* private bertujuan untuk menjaga keamanan data dan menerapkan konsep enkapsulasi dalam pemrograman berorientasi objek.

Pada bagian konstruktor, program digunakan untuk menginisialisasi nilai atribut ketika objek mahasiswa dibuat. Konstruktor menerima tiga parameter, yaitu nama_2027, nim_2027, dan prodi_2027, kemudian menyimpan nilai tersebut ke dalam atribut kelas menggunakan this. Dengan adanya konstruktor, proses

pembuatan objek menjadi lebih mudah karena data mahasiswa dapat langsung dimasukkan saat objek dibuat.

Program juga memiliki beberapa method getter dan setter. Method getter seperti `getNama_2027()`, `getNim_2027()`, dan `getProdi_2027()` digunakan untuk mengambil nilai atribut dari objek mahasiswa. Sedangkan method setter seperti `setNama_2027()`, `setNim_2027()`, dan `setProdi_2027()` digunakan untuk mengubah nilai atribut. Penggunaan getter dan setter merupakan bagian dari penerapan prinsip enkapsulasi agar akses terhadap data dapat dikontrol dengan baik.

Pada bagian akhir terdapat method `toString()` yang di-*override* dari kelas `Object`. Method ini digunakan untuk mengembalikan nilai berupa nama mahasiswa ketika objek ditampilkan atau dicetak. Dengan adanya method `toString()`, objek mahasiswa dapat langsung ditampilkan dalam bentuk teks yang lebih sederhana dan mudah dibaca.

```
import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.util.ArrayList;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;

public class SortingGUI_2511532027 extends JFrame {
    private static final long serialVersionUID = 1L;

    // Komponen GUI
    private JTextField namaField_2027;
    private JTextField nimField_2027;
    private JTextField prodiField_2027;
    private JButton tambahButton_2027;
    private JButton sortingButton_2027;
    private JComboBox<String> comboBox_2027;
```

```

private JTextArea outputArea_2027;

// ArrayList utama
private ArrayList<Mahasiswa_2511532027> dataMahasiswa_2027;

// Constructor
public SortingGUI_2511532027() {
    setTitle("Visualisasi Sorting Mahasiswa");
    setSize(850, 550);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());
    dataMahasiswa_2027 = new ArrayList<>();

    // ===== PANEL INPUT =====

    JPanel inputPanel_2027 = new JPanel(new FlowLayout());
    namaField_2027 = new JTextField(10);
    nimField_2027 = new JTextField(10);
    prodiField_2027 = new JTextField(10);
    tambahButton_2027 = new JButton("Tambah Data");
    sortingButton_2027 = new JButton("Mulai Sorting");
    comboBox_2027 = new JComboBox<>();
    comboBox_2027.addItem("Insertion Sort");
    comboBox_2027.addItem("Selection Sort");
    comboBox_2027.addItem("Bubble Sort");

    inputPanel_2027.add(new JLabel("Nama"));
    inputPanel_2027.add(namaField_2027);
    inputPanel_2027.add(new JLabel("NIM"));
    inputPanel_2027.add(nimField_2027);
    inputPanel_2027.add(new JLabel("Prodi"));
    inputPanel_2027.add(prodiField_2027);
    inputPanel_2027.add(tambahButton_2027);
    inputPanel_2027.add(comboBox_2027);
    inputPanel_2027.add(sortingButton_2027);

    // ===== TEXT AREA =====

```



```

        outputArea_2027 = new JTextArea();
        outputArea_2027.setEditable(false);
        JScrollPane scrollPane_2027 = new JScrollPane(outputArea_2027);
        add(inputPanel_2027, BorderLayout.NORTH);
        add(scrollPane_2027, BorderLayout.CENTER);

        // ===== EVENT =====

        // Tambah data
        tambahButton_2027.addActionListener(e_2027 ->
tambahData_2027());

        // Mulai sorting
        sortingButton_2027.addActionListener(e_2027 ->
mulaiSorting_2027());
    }

    // ===== TAMBAH DATA =====

    private void tambahData_2027() {
        String nama_2027 = namaField_2027.getText();
        String nim_2027 = nimField_2027.getText();
        String prodi_2027 = prodiField_2027.getText();

        // Validasi
        if (nama_2027.isEmpty() || nim_2027.isEmpty() ||
prodi_2027.isEmpty()) {
            JOptionPane.showMessageDialog( this, "Data belum lengkap!");
            return;
        }

        // Tambah ke ArrayList
        Mahasiswa_2511532027 mhs_2027 = new
Mahasiswa_2511532027(nama_2027, nim_2027, prodi_2027);
        dataMahasiswa_2027.add(mhs_2027);
        tampilkanData_2027();
    }

```

```

        // Kosongkan field
        namaField_2027.setText("");
        nimField_2027.setText("");
        prodiField_2027.setText("");
    }

    // ===== TAMPILKAN DATA =====

    private void tampilkanData_2027() {
        outputArea_2027.setText("");
        outputArea_2027.append("==== DATA MAHASISWA =====\n\n");
        for (Mahasiswa_2511532027 mhs_2027 : dataMahasiswa_2027) {
            outputArea_2027.append(mhs_2027.getNama_2027() + " | "
                + mhs_2027.getNim_2027() + " | "
                + mhs_2027.getProdi_2027() + "\n");
        }
    }

    // ===== MULAI SORTING =====

    private void mulaiSorting_2027() {
        if (dataMahasiswa_2027.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Data mahasiswa
kosong!");
            return;
        }

        // Membuat salinan data lama
        ArrayList<Mahasiswa_2511532027> copyData_2027 = new
ArrayList<>();
        for (Mahasiswa_2511532027 mhs_2027 : dataMahasiswa_2027) {
            copyData_2027.add(new Mahasiswa_2511532027(
                mhs_2027.getNama_2027(),
                mhs_2027.getNim_2027(),
                mhs_2027.getProdi_2027()));
        }
    }

```

```

        String pilihan_2027 =
comboBox_2027.getSelectedItem().toString();
        outputArea_2027.append("\n\n=====\n");
        outputArea_2027.append("DATA AWAL : " +
arrayToString_2027(copyData_2027) + "\n\n");

        // INSERTION SORT
        if (pilihan_2027.equals("Insertion Sort")) {
            outputArea_2027.append("=== INSERTION SORT ===\n\n");
            insertionSort_2027(copyData_2027);
        }

        // SELECTION SORT
        else if (pilihan_2027.equals("Selection Sort")) {
            outputArea_2027.append("=== SELECTION SORT ===\n\n");
            selectionSort_2027(copyData_2027);
        }

        // BUBBLE SORT
        else {
            outputArea_2027.append("=== BUBBLE SORT ===\n\n");
            bubbleSort_2027(copyData_2027);
        }

        outputArea_2027.append("\nHASIL AKHIR : " +
arrayToString_2027(copyData_2027) + "\n");
        outputArea_2027.append("=====\n");
    }

    // ===== INSERTION SORT =====

    private void insertionSort_2027(
        ArrayList<Mahasiswa_2511532027> data_2027) {
        for (int i_2027 = 1; i_2027 < data_2027.size(); i_2027++) {
            Mahasiswa_2511532027 key_2027 = data_2027.get(i_2027);
            int j_2027 = i_2027 - 1;
            while (j_2027 >= 0 && data_2027.get(j_2027).getNama_2027()
                .compareToIgnoreCase(key_2027.getNama_2027()) > 0) {
                data_2027.set(j_2027 + 1, data_2027.get(j_2027));
            }
        }
    }

```

```

        j_2027--;
    }
    data_2027.set(j_2027 + 1, key_2027);
    outputArea_2027.append("Langkah " + i_2027 + " : "
        + arrayToString_2027(data_2027) + "\n");
    }
}

// ===== SELECTION SORT =====

private void selectionSort_2027(
    ArrayList<Mahasiswa_2511532027> data_2027) {
    for (int i_2027 = 0; i_2027 < data_2027.size() - 1; i_2027++) {
        int minIndex_2027 = i_2027;
        for (int j_2027 = i_2027 + 1; j_2027 < data_2027.size();
j_2027++) {
            if
(data_2027.get(j_2027).getNama_2027().compareToIgnoreCase(
data_2027.get(minIndex_2027).getNama_2027()) < 0) {
                minIndex_2027 = j_2027;
            }
        }
        Mahasiswa_2511532027 temp_2027 = data_2027.get(i_2027);
        data_2027.set(i_2027, data_2027.get(minIndex_2027));
        data_2027.set(minIndex_2027, temp_2027);
        outputArea_2027.append("Pass " + (i_2027 + 1) + " : "
            + arrayToString_2027(data_2027) + "\n");
    }
}

// ===== BUBBLE SORT =====

private void bubbleSort_2027(
    ArrayList<Mahasiswa_2511532027> data_2027) {
    for (int i_2027 = 0; i_2027 < data_2027.size() - 1; i_2027++) {
        for (int j_2027 = 0; j_2027 < data_2027.size() - i_2027 - 1;
j_2027++) {

```

```

        if
(data_2027.get(j_2027).getNama_2027().compareToIgnoreCase(
                        data_2027.get(j_2027 +
1).getNama_2027()) > 0) {
            Mahasiswa_2511532027 temp_2027 =
data_2027.get(j_2027);
            data_2027.set(j_2027, data_2027.get(j_2027 + 1));
            data_2027.set(j_2027 + 1, temp_2027);
        }
    }
    outputArea_2027.append("Pass " + (i_2027 + 1) + " : "
        + arrayToString_2027(data_2027) + "\n");
}
}

// ===== ARRAYLIST KE STRING =====

private String arrayToString_2027(
    ArrayList<Mahasiswa_2511532027> data_2027) {
    String hasil_2027 = "[";
    for (int i_2027 = 0; i_2027 < data_2027.size(); i_2027++) {
        hasil_2027 += data_2027.get(i_2027).getNama_2027();
        if (i_2027 < data_2027.size() - 1) {
            hasil_2027 += ", ";
        }
    }
    hasil_2027 += "]";
    return hasil_2027;
}

// ===== MAIN =====

public static void main(String[] args) {
    SortingGUI_2511532027 gui_2027 = new SortingGUI_2511532027();
    gui_2027.setVisible(true);
}
}

```

Kode program tersebut merupakan aplikasi GUI yang dibuat menggunakan *library* Swing untuk melakukan visualisasi proses *sorting* data mahasiswa. Kelas `SortingGUI_2511532027` merupakan turunan dari kelas `JFrame`, sehingga program dapat menampilkan jendela aplikasi interaktif. Program ini digunakan untuk memasukkan data mahasiswa berupa nama, NIM, dan program studi, kemudian melakukan proses pengurutan data menggunakan algoritma *Insertion Sort*, *Selection Sort*, atau *Bubble Sort*. Seluruh data mahasiswa disimpan dalam sebuah *ArrayList* bernama `dataMahasiswa_2027` yang berisi objek dari kelas `Mahasiswa_2511532027`.

Pada bagian konstruktor, program mengatur tampilan utama GUI seperti judul *frame*, ukuran jendela, posisi tampilan, dan *layout* menggunakan `BorderLayout`. Selanjutnya dibuat berbagai komponen GUI seperti `TextField` untuk input data, `Button` untuk tombol aksi, `ComboBox` untuk memilih algoritma *sorting*, serta `TextArea` untuk menampilkan hasil proses *sorting*. Komponen-komponen tersebut dimasukkan ke dalam `JPanel` menggunakan `FlowLayout`, kemudian ditambahkan ke *frame* utama. Program juga menambahkan *event listener* pada tombol “Tambah Data” dan “Mulai Sorting” agar dapat menjalankan method tertentu saat tombol ditekan.

Method `tambahData_2027()` digunakan untuk mengambil input dari pengguna, kemudian memvalidasi apakah seluruh *field* telah diisi. Jika ada *field* yang kosong, program akan menampilkan pesan kesalahan menggunakan `JOptionPane`. Jika data valid, maka program membuat objek mahasiswa baru dan menyimpannya ke dalam *ArrayList*. Setelah data berhasil ditambahkan, method `tampilkanData_2027()` dipanggil untuk menampilkan seluruh data mahasiswa ke area output. Method ini juga membersihkan *field* input agar pengguna dapat memasukkan data berikutnya dengan lebih mudah.

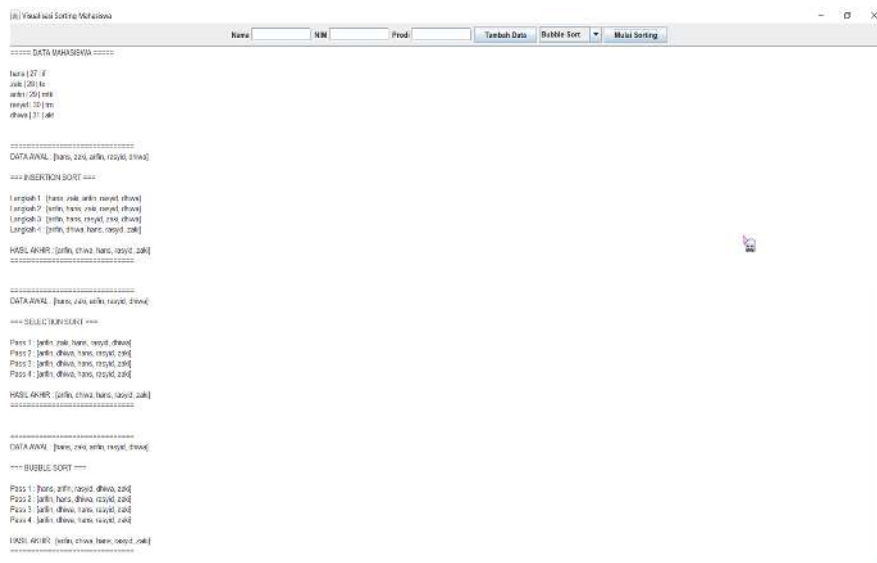
Method `mulaiSorting_2027()` digunakan untuk menjalankan proses *sorting* sesuai algoritma yang dipilih pada `ComboBox`. Sebelum *sorting* dilakukan, program membuat salinan data mahasiswa ke dalam `copyData_2027` agar data asli tetap tersimpan dan tidak berubah. Program kemudian menampilkan data awal sebelum *sorting* dimulai. Setelah itu, program memeriksa algoritma yang dipilih pengguna, lalu memanggil method `insertionSort_2027()`, `selectionSort_2027()`, atau

bubbleSort_2027() sesuai pilihan. Setelah proses selesai, hasil akhir pengurutan ditampilkan pada area output.

Method insertionSort_2027() menerapkan algoritma *Insertion Sort* dengan cara mengambil satu elemen sebagai *key*, lalu membandingkannya dengan elemen sebelumnya hingga posisi yang tepat ditemukan. Selama proses berlangsung, setiap langkah pengurutan ditampilkan pada JTextArea. Method selectionSort_2027() menggunakan algoritma *Selection Sort* dengan mencari elemen terkecil pada setiap iterasi, kemudian menukarkannya dengan posisi saat ini. Sedangkan method bubbleSort_2027() merupakan algoritma *Bubble Sort* dengan membandingkan dua elemen yang berdekatan dan menukarnya jika urutannya salah. Ketiga method tersebut menampilkan proses *sorting* secara bertahap agar pengguna dapat memahami alur algoritma dengan lebih jelas.

Selain itu, program memiliki method arrayToString_2027() yang digunakan untuk mengubah isi *ArrayList* menjadi bentuk teks agar dapat ditampilkan pada JTextArea. Method ini menggabungkan nama-nama mahasiswa ke dalam format *array* seperti [Andi, Budi, Caca]. Pada bagian akhir program terhadap method main() yang berfungsi sebagai titik awal eksekusi program. Method ini membuat objek SortingGUI_2511532027 dan menampilkan GUI ke layar menggunakan method setVisible(true).

Hasil / Output :



Gambar 2.5

BAB 3

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma *sorting* seperti *Insertion Sort*, *Selection Sort*, dan *Bubble Sort* merupakan metode dasar pengurutan data yang memiliki cara kerja yang berbeda-beda dalam menyusun data secara *ascending*. *Insertion Sort* menyisipkan elemen pada posisi yang tepat, *Selection Sort* mencari nilai terkecil pada setiap iterasi, dan *Bubble Sort* membandingkan elemen yang berdekatan. Melalui praktikum ini, mahasiswa dapat memahami proses kerja masing-masing algoritma, mengimplementasikannya ke dalam program Java, serta mengetahui kelebihan dan kekurangan dari setiap metode *sorting*. Selain itu, pembuatan program berbasis GUI juga membantu memvisualisasikan proses pengurutan sehingga algoritma lebih mudah dipahami.

Dalam pelaksanaan praktikum selanjutnya, disarankan agar mahasiswa lebih banyak melakukan percobaan menggunakan jumlah data yang berbeda-beda untuk melihat perbandingan kinerja masing-masing algoritma *sorting*. Mahasiswa juga dapat mencoba mengembangkan program dengan menambahkan fitur animasi, pilihan metode *sorting*, maupun pengurutan secara *descending* agar program lebih interaktif. Selain itu, pemahaman mengenai logika algoritma dan struktur program perlu terus dilatih agar kemampuan pemrograman serta analisis terhadap efisiensi algoritma dapat meningkat dengan lebih baik.

DAFTAR PUSTAKA

- [1] <https://www.geeksforgeeks.org/dsa/comparison-among-bubble-sort-selection-sort-and-insertion-sort/> [Diakses 19 Mei 2026]
- [2] <https://openjournal.unpam.ac.id/index.php/informatika/article/view/38800> [Diakses 19 Mei 2026]
- [3] <https://ejournal.unjaya.ac.id/index.php/teknomatika/article/view/363> [Diakses 19 Mei 2026]
- [4] <https://conference.ut.ac.id/index.php/saintek/article/view/2617> [Diakses 19 Mei 2026]

LAMPIRAN

Gambar 2.1	15
Gambar 2.2	16
Gambar 2.3	17
Gambar 2.4	18
Gambar 2.5	28